

PoC: Koexistenz von Envoy und Apache APISIX im Kubernetes-Zielbild

Kurzfassung

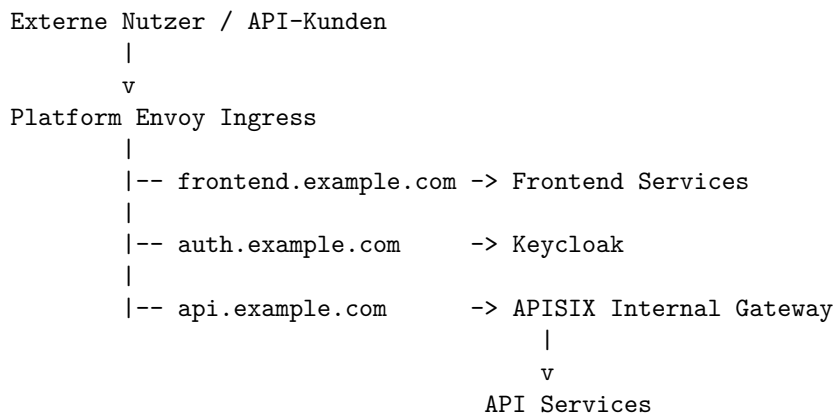
Im Zielbild bleibt Envoy der plattformseitig kontrollierte öffentliche Ingress. Apache APISIX wird zusätzlich als interner API-Gateway-Layer für API-spezifische Anforderungen eingesetzt.

APISIX ersetzt Envoy nicht global. Stattdessen wird APISIX nur für ausgewählte API-Workloads verwendet, bei denen API-Management-Funktionen wie Authentifizierung, Rate Limiting, Request/Response Transformationen, Routing Policies und API-spezifische Observability benötigt werden.

Frontend-Anwendungen bleiben hinter dem bestehenden Envoy Ingress. API-Endpunkte werden über Envoy erreicht und anschließend intern an APISIX weitergeleitet.

Keycloak wird als neue zentrale Identity-Lösung vorgesehen. Die alte Auth-Lösung wird nicht nach Kubernetes migriert, da sie abgekündigt beziehungsweise nicht mehr zukunftsfähig ist.

Zielarchitektur



Rollenverteilung

Komponente	Verantwortung
Envoy	Öffentlicher Ingress, TLS, DNS-Anbindung, Edge Security, zentrale Plattformkontrolle
APISIX	API Gateway, API Auth Enforcement, Rate Limiting, Routing, Transformations, API Policies
Keycloak	Identity Provider, OAuth2/OIDC, Token-Ausstellung für Nutzer, API-Kunden und Services
Backend Services	Fachliche Autorisierung, kundenspezifische Ressourcenprüfung und Business-Logik

Kernaussage

APISIX wird nicht als zweite öffentliche Eingangstür betrieben. Envoy bleibt der einzige externe Ingress-Punkt. APISIX wird als interner API-Gateway-Layer hinter Envoy eingesetzt.

Damit entsteht keine zweite öffentliche Angriffsfläche. Die externe Plattformkontrolle bleibt bei Envoy, während APISIX API-spezifische Gateway-Funktionen übernimmt.

Keycloak stellt JWTs für externe Kunden, Benutzer und interne Service-Kommunikation aus. APISIX validiert und erzwingt API-nahe Policies. Backend Services verwenden dieselben JWTs beziehungsweise über Token Exchange abgeleitete JWTs für fachliches RBAC und Zugriff auf kundeneigene Ressourcen.

Direkte API-Kunden

Direkte API-Kunden authentifizieren sich gegen Keycloak und rufen anschließend die APIs über den bestehenden Envoy Ingress auf.

Ablauf:

1. API-Kunde fordert Token bei Keycloak an
2. API-Kunde ruft `api.example.com` über Envoy auf
3. Envoy leitet den Request an APISIX weiter
4. APISIX validiert das Keycloak Token
5. APISIX routet zum passenden Backend Service
6. Backend Service prüft fachliches RBAC und kundeneigene Ressourcen

Beispiel:

```
POST https://auth.example.com/realms/<realm>/protocol/openid-connect/token
GET  https://api.example.com/orders
Authorization: Bearer <access_token>
```

Envoy muss dabei die relevanten Auth-Header erhalten, insbesondere:

```
Authorization
X-API-Key, falls für Übergangsszenarien benötigt
Host, falls APISIX Host-basiertes Routing verwendet
```

Envoy sollte Forwarding-Header kontrolliert setzen oder überschreiben:

```
X-Forwarded-For
X-Forwarded-Proto
X-Request-ID
```

APISIX darf diese Header nur aus vertrauenswürdigen Requests vom Envoy Ingress akzeptieren.

Keycloak

Keycloak wird als neue zentrale Identity-Lösung vorgesehen, da die alte Auth-Lösung nicht weitergeführt und nicht nach Kubernetes migriert wird.

Empfohlenes Zielbild:

```
Namespace: identity
Service:   keycloak
Ingress:   Envoy, z. B. auth.example.com
Database:  dedizierte PostgreSQL-Datenbank
```

Keycloak stellt Tokens für folgende Szenarien aus:

Szenario	Empfohlener Flow
API-Kunde / Maschine-zu-Maschine	Client Credentials
Frontend mit Benutzerlogin	Authorization Code + PKCE
Interne Service-Kommunikation	Token Exchange / On-Behalf-Of oder Service Account Token
Übergang für Legacy-Kunden	API Key, zeitlich begrenzt

Wichtig: Tokens müssen passende **audience**, **scope** oder Rollen-Claims enthalten. Ein gültiges Token darf nicht automatisch Zugriff auf alle APIs oder alle kundeneigenen Ressourcen bedeuten.

JWT-basiertes Zielbild für externe und interne Kommunikation

Die Zielarchitektur setzt auf JWTs als gemeinsamen Identitäts- und Autorisierungskontext.

Keycloak stellt Tokens für externe API-Kunden, Benutzer-Frontends und interne Service-Kommunikation aus. Diese Tokens enthalten Claims für Mandant, Kunde, Rollen, Scopes und Ziel-Audience.

JWTs werden sowohl am API-Gateway als auch innerhalb der Services verwendet, um Zugriff auf kundeneigene Ressourcen und fachliche Berechtigungen konsistent zu prüfen.

Verwendung der JWTs

Kommunikationspfad	JWT-Verwendung
Externer API-Kunde -> Envoy -> APISIX -> API Service	Customer Token wird von APISIX validiert und vom API Service für fachliches RBAC genutzt
Browser User -> Frontend -> API	User Token wird über APISIX validiert und im Backend für fachliche Autorisierung genutzt
Service A -> Service B	Service- oder On-Behalf-Of-Token wird intern weitergegeben und von Service B validiert
Backend Service -> kundenspezifische Ressource	JWT Claims bestimmen Mandant, Kunde, Rollen und erlaubte Ressourcen

Token-Inhalte

JWTs müssen ausreichend Kontext für Gateway- und Service-seitige Entscheidungen enthalten.

Relevante Claims:

<code>iss</code>	Issuer, z. B. Keycloak Realm
<code>sub</code>	Benutzer oder Service Account
<code>aud</code>	Zielsystem / API Audience
<code>azp</code>	Authorized Party / aufrufender Client
<code>scope</code>	Erlaubte API-Scopes
<code>roles</code>	Rollen für fachliches RBAC
<code>tenant_id</code>	Mandant / Kunde
<code>customer_id</code>	Kundenzuordnung
<code>resource_access</code>	Keycloak Client Rollen
<code>exp</code>	Ablaufzeit
<code>iat</code>	Ausstellungszeitpunkt
<code>jti</code>	Token ID für Nachvollziehbarkeit

Wichtig: Ein gültiges Token reicht nicht aus. Services müssen prüfen, ob `aud`, `scope`, Rollen und Mandantenkontext zum angefragten Use Case passen.

RBAC-Modell

Keycloak dient als zentrale Quelle für Identitäts-, Rollen- und Scope-Informationen.

APISIX übernimmt Gateway-nahe Prüfungen:

- Ist das Token gültig?
- Ist der Issuer vertrauenswürdig?
- Passt die Audience zur API?
- Ist der benötigte Scope vorhanden?
- Greift eine Rate-Limit- oder API-Policy?

Backend Services übernehmen fachliche Autorisierung:

- Darf dieser Benutzer oder Service diese konkrete Ressource lesen oder ändern?
- Gehört die Ressource zum `tenant_id` / `customer_id` aus dem Token?
- Reicht die Rolle für die angefragte Operation?
- Ist der Zugriff im aktuellen Geschäftsprozess erlaubt?

Damit bleibt APISIX für API-Policy Enforcement zuständig, während fachliche RBAC-Entscheidungen in den Services verbleiben.

Interne Service-Kommunikation

Für interne Service-zu-Service-Kommunikation sollen JWTs ebenfalls verwendet werden.

Dabei muss zwischen drei Konzepten unterschieden werden:

Konzept	Bedeutung
User/Customer Token	Token repräsentiert einen externen Benutzer oder API-Kunden
Service Token	Token repräsentiert einen technischen Service ohne Benutzerkontext
On-Behalf-Of Token	Token repräsentiert einen Service, der im Auftrag eines Benutzers oder Kunden handelt

Diese Unterscheidung ist wichtig, weil nicht jeder interne Request denselben Sicherheitskontext hat. Ein nächtlicher Batch-Job, ein API-Request eines Kunden und ein Backend-Call im Auftrag eines eingeloggten Benutzers sind fachlich unterschiedliche Situationen.

Token Forwarding

Ein Service gibt das ursprüngliche Benutzer- oder Kundentoken an nachgelagerte Services weiter.

Geeignet für:

- fachliche Aktionen im Namen eines Benutzers
- durchgängige Auditierbarkeit
- Ressourcenzugriff basierend auf Benutzer-/Kundenkontext

Risiken:

- Token wird breiter im Cluster verteilt
- Downstream Services sehen eventuell mehr Claims als nötig
- Token-Lebensdauer und Audience müssen sauber begrenzt sein
- Das ursprüngliche Token ist eventuell nicht für den Downstream-Service als Audience ausgestellt
- Es kann unklar werden, welcher Service welche Aktion tatsächlich ausgelöst hat

Token Forwarding ist verständlich und einfach, sollte aber nicht das Standardmuster für kritische Serviceketten sein. Es ist besonders dann riskant, wenn ein Token mit breiter Audience an viele Services weitergereicht wird.

Service Tokens

Ein Service Token repräsentiert einen technischen Client oder Workload. Es steht nicht für einen menschlichen Benutzer und nicht direkt für einen externen Kunden.

Typische Beispiele:

- Scheduler ruft Billing Service auf
- Import Service verarbeitet eine Datei
- Notification Service verschickt systemgenerierte Nachrichten
- Reporting Service liest aggregierte Daten

Ein Service Token wird typischerweise über den Client Credentials Flow ausgestellt:

```
Service A authentifiziert sich bei Keycloak
Keycloak stellt Token mit sub=service-a und aud=service-b aus
Service A ruft Service B mit diesem Token auf
Service B validiert Service-Identität, Audience, Scopes und Rollen
```

Beispielhafte Claims:

```
sub    = service-account-import-service
azp    = import-service
aud    = billing-service
scope  = billing:write
roles  = service_importer
```

Service Tokens eignen sich für technische Aktionen, die nicht im Namen eines konkreten Benutzers oder Kunden stattfinden.

Wichtig: Service Tokens dürfen nicht als pauschaler Generalschlüssel verwendet werden. Auch Service Accounts brauchen spezifische Audiences, Scopes und Rollen.

On-Behalf-Of Tokens

Ein On-Behalf-Of Token repräsentiert eine Weitergabe des ursprünglichen Benutzer- oder Kundenkontexts, aber mit einer neuen, engeren Ziel-Audience für den nächsten Service.

Ein Service tauscht ein eingehendes Token gegen ein neues Token für den konkreten Downstream-Service aus.

Geeignet für:

- stärkere Begrenzung der Audience
- bessere Least-Privilege-Umsetzung
- klare Service-zu-Service-Vertrauensgrenzen
- Auditierbarkeit von Benutzer/Kunde und aufrufendem Service
- Downstream Services, die nur Tokens für ihre eigene Audience akzeptieren sollen

Beispiel:

```
User/Customer Token -> Service A
Service A tauscht Token bei Keycloak
Service A erhält Token mit aud=service-b
Service A ruft Service B mit neuem Token auf
Service B validiert aud=service-b und Claims
```

Das neue Token kann weiterhin den ursprünglichen Benutzer oder Kunden enthalten, aber zusätzlich den aufrufenden Service sichtbar machen.

Beispielhafte Claims:

```
sub          = user-123 oder customer-client-456
azp          = service-a
aud          = service-b
scope        = orders:read
tenant_id    = customer-123
act          = service-a
```

Der genaue Claim-Name für den Actor-Kontext hängt von der Identity-Plattform und Konfiguration ab. Das Ziel ist aber immer gleich: Service B soll erkennen können, dass der Request fachlich im Kontext von Benutzer/Kunde X erfolgt, technisch aber von Service A ausgeführt wurde.

Empfehlung für die Zielarchitektur: Token Exchange / On-Behalf-Of bevorzugen, wo möglich. Token Forwarding kann als Übergang oder für einfache Pfade genutzt werden.

Entscheidungsmatrix für interne Tokens

Situation	Empfohlenes Muster
Benutzer klickt im Frontend und Backend ruft weitere Services	On-Behalf-Of Token
Externer API-Kunde ruft API auf und API ruft weitere Services	On-Behalf-Of Token oder streng begrenztes Token Forwarding
Technischer Job ohne Benutzerkontext	Service Token
Einfacher Übergangspfad in Legacy-Systeme	Token Forwarding mit enger Laufzeit und klarer Audience
Zugriff auf kundeneigene Ressourcen	User/Customer Kontext muss erhalten bleiben, bevorzugt über On-Behalf-Of
Hochkritische Servicekette	Token Exchange / On-Behalf-Of mit spezifischer Audience pro Downstream-Service

Warum On-Behalf-Of wichtig ist

Ohne On-Behalf-Of entstehen oft zwei problematische Extreme:

- Das ursprüngliche Kundentoken wird durch viele Services gereicht und ist zu breit verwendbar.
- Ein interner Service verwendet nur sein eigenes Service Token und verliert den Kunden- oder Benutzerkontext.

Beide Varianten sind für kundeneigene Ressourcen ungünstig.

On-Behalf-Of löst dieses Problem, indem beide Informationen erhalten bleiben:

Wer ist der fachliche Akteur?

-> Benutzer oder API-Kunde

Welcher Service führt den nächsten technischen Schritt aus?

-> aufrufender Backend Service

Für welchen Zielservice ist dieses Token gedacht?

-> konkrete Audience des Downstream-Service

Damit kann ein Downstream-Service sauber prüfen:

- Ist das Token für mich bestimmt?
- Welcher Service ruft mich auf?
- Im Auftrag welches Benutzers oder Kunden passiert der Zugriff?
- Welche Scopes und Rollen gelten?
- Passt der Mandantenkontext zur angefragten Ressource?

Beispiel: Kundenressource mit On-Behalf-Of

1. API-Kunde ruft GET /orders über Envoy und APISIX auf
2. APISIX validiert das Customer Token
3. Order API benötigt Daten vom Customer Profile Service
4. Order API tauscht das Customer Token gegen ein Token für customer-profile-service
5. Customer Profile Service validiert aud=customer-profile-service
6. Customer Profile Service prüft tenant_id/customer_id und scope
7. Customer Profile Service liefert nur Daten des erlaubten Kunden zurück

Beispielhafte Prüfungen im Downstream-Service:

aud enthält customer-profile-service
scope enthält customer-profile:read
tenant_id passt zur angefragten Ressource
azp/act zeigt den aufrufenden Service
iss ist der erwartete Keycloak Realm
exp ist gültig

Anti-Patterns

- Ein internes Service Token mit Zugriff auf alle Kundenressourcen.
- Ein externes Kundentoken wird unverändert durch alle Services gereicht.
- Services prüfen nur, ob ein JWT gültig ist, aber nicht aud, scope, Rollen oder Mandant.
- Kundenzuordnung wird aus einem frei gesetzten HTTP Header statt aus vertrauenswürdigen Claims abgeleitet.
- APISIX übernimmt fachliches RBAC, obwohl nur der Backend Service die Ressource und Geschäftsregel kennt.
- Logs enthalten vollständige Access Tokens.
- Ein Token ist für mehrere APIs und Services gleichzeitig gültig, obwohl nur ein Zielservice benötigt wird.

Mandanten- und Ressourcenautorisierung

Da kundeneigene Ressourcen geschützt werden müssen, muss der Mandantenkontext verbindlich aus dem Token oder aus einer vertrauenswürdigen Zuordnung abgeleitet werden.

Beispielprüfung in einem Backend Service:

Request: GET /customers/123/orders

Token Claims:

tenant_id = customer-123
scope = orders:read
roles = customer_admin

Service-Prüfung:

- Gehört /customers/123 zum tenant_id aus dem Token?
- Enthält scope orders:read?
- Reicht role customer_admin für diese Aktion?

Wenn eine Ressource nicht zum Mandantenkontext des Tokens passt, muss der Zugriff verweigert werden, auch wenn das Token technisch gültig ist.

Kubernetes Gateway API

APISIX kann als zusätzliche Gateway API Implementierung über eine eigene `GatewayClass` betrieben werden.

Beispiel:

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: apisix
spec:
  controllerName: apisix.apache.org/apisix-ingress-controller
```

Die Plattform kann weiterhin Envoy als Default GatewayClass betreiben. APISIX wird nur von Workloads genutzt, die explizit diese GatewayClass verwenden.

Abgrenzung zu internem Service Routing

Die Einführung von APISIX betrifft nur explizit angebotenen API-Ingress-Traffic.

Normales internes Kubernetes Service Routing bleibt unverändert:

```
frontend -> orders-service.namespace.svc.cluster.local
service-a -> service-b.namespace.svc.cluster.local
```

APISIX beeinflusst diesen Traffic nicht automatisch.

Falls die Plattform Envoy zusätzlich als Service-Mesh oder East-West-Dataplane verwendet, bleibt diese Funktion weiterhin unabhängig von APISIX bestehen. APISIX ist in diesem Vorschlag ein API-Gateway, kein Ersatz für Service Mesh Routing.

JWT-basierte Autorisierung kann trotzdem cluster-intern verwendet werden. Diese Autorisierung findet dann in den aufgerufenen Services oder in gemeinsam genutzten Auth-Bibliotheken statt, nicht durch eine automatische Änderung des Kubernetes Service Routings.

Auswirkungen auf APISIX und Envoy

Envoy bleibt weiterhin der öffentliche Ingress und leitet Auth-Header unverändert an APISIX weiter.

APISIX validiert externe JWTs und kann grundlegende API-Policies erzwingen. APISIX sollte jedoch nicht die einzige Quelle für fachliche Autorisierung sein.

Interne Service-Kommunikation läuft weiterhin über Kubernetes Services. JWT-basierte Autorisierung wird dabei von den aufgerufenen Services oder einer gemeinsamen Auth-Library validiert.

APISIX verändert internes Service Routing nicht automatisch.

Sicherheitsmodell

APISIX wird nicht öffentlich exponiert.

Mindestanforderungen:

- Envoy bleibt einziger öffentlicher Ingress.
- APISIX Service ist nur aus dem Envoy-/Ingress-Kontext erreichbar.
- APISIX Admin API ist nicht öffentlich erreichbar.
- APISIX Dashboard ist deaktiviert oder nur intern/VPN erreichbar.
- etcd ist nur intern für APISIX erreichbar.

- NetworkPolicies beschränken Zugriff auf APISIX, etcd und Keycloak.
- Auth-Header werden von Envoy nicht entfernt.
- Client-gelieferte Forwarding-Header werden durch Envoy normalisiert oder überschrieben.
- APISIX validiert JWTs über Keycloak JWKS/OIDC Discovery.
- Backend Services prüfen fachliche Berechtigungen weiterhin selbst oder validieren Tokens zusätzlich.
- Tokens müssen kurze Lebensdauer haben.
- Services validieren Signatur, `iss`, `aud`, `exp` und relevante Scopes/Rollen.
- `aud` muss möglichst spezifisch sein.
- Kundenzuordnung darf nicht aus frei manipulierbaren Headern stammen.
- Sensitive Claims werden minimiert.
- Service Accounts in Keycloak müssen klar getrennt sein.
- Token Exchange ist für kritische Serviceketten gegenüber blindem Token Forwarding zu bevorzugen.
- Logs dürfen keine vollständigen Tokens enthalten.
- Zentraler JWKS Cache oder robuste JWKS-Verfügbarkeit muss eingeplant werden.

PoC-Ziele

Der PoC soll zeigen, dass Envoy und APISIX ohne Konflikt koexistieren können.

Umfang

- APISIX Ingress Controller installieren.
- Eigene `GatewayClass` für APISIX anlegen.
- APISIX nur intern erreichbar machen.
- Envoy Route für `api.example.com` auf APISIX zeigen lassen.
- Keycloak als OIDC Provider anbinden.
- Beispiel-API hinter APISIX veröffentlichen.
- JWT-Validierung über Keycloak testen.
- Rate Limit oder andere APISIX Policy aktivieren.
- Beispiel für internes Service-to-Service JWT validieren.
- Nachweisen, dass bestehende Envoy Ingresses unverändert funktionieren.
- Nachweisen, dass internes Service Routing unverändert bleibt.
- Nachweisen, dass kundeneigene Ressourcen über Claims wie `tenant_id`, `customer_id`, `scope` und Rollen geschützt werden können.

Nicht-Ziele

- Kein globaler Ersatz von Envoy.
- Keine Änderung des plattformseitigen Standard-Ingress.
- Keine Migration aller Workloads auf APISIX.
- Kein Ersatz eines bestehenden Service Mesh.
- Keine öffentliche Exposition der APISIX Admin API.
- Keine vollständige Neumodellierung aller fachlichen Berechtigungen im Gateway.

Erfolgskriterien

Der PoC gilt als erfolgreich, wenn:

- Envoy weiterhin der einzige öffentliche Ingress bleibt.
- `frontend.example.com` unverändert über Envoy funktioniert.
- `api.example.com` über Envoy an APISIX weitergeleitet wird.
- APISIX ein Keycloak-issued JWT erfolgreich validiert.
- APISIX eine API Policy wie Rate Limiting oder Routing Policy anwenden kann.
- APISIX Admin API, Dashboard und etcd nicht öffentlich erreichbar sind.
- Bestehendes internes Kubernetes Service Routing unverändert bleibt.
- Die Plattform mehrere Gateway API Implementierungen sauber unterscheiden kann.
- Ein Backend Service fachliches RBAC auf Basis von JWT Claims validieren kann.
- Ein Service-to-Service-Aufruf mit weitergegebenem oder getauschtem JWT demonstriert werden kann.
- Ein Zugriff auf eine nicht zum Mandanten passende Ressource abgelehnt wird.

Quellen und Belege

Dieser Abschnitt ordnet die zentralen Aussagen des Dokuments belastbaren Quellen zu. Die Architektur selbst ist eine Zielbild-Ableitung aus diesen Standards und Produktdokumentationen, keine 1:1 aus einer einzelnen Quelle übernommene Referenzarchitektur.

Aussage	Belege
Gateway API erlaubt unterschiedliche Gateway-Implementierungen über <code>GatewayClass</code> .	Kubernetes Gateway API: <code>GatewayClass</code> beschreibt <code>GatewayClass</code> als clusterweite Klasse von Gateways und entkoppelt Controller/Implementierung vom Nutzer.
Ein Gateway bindet Routes an konkrete Listener und Backend Services.	Kubernetes Gateway API: Gateway beschreibt <code>Gateway</code> , <code>Listener</code> und <code>GatewayClassName</code> als Verbindung zwischen Infrastruktur und Route-Konfiguration.
APISIX kann als Kubernetes Gateway API Implementierung betrieben werden.	Apache APISIX Ingress Controller: Gateway API dokumentiert Gateway API Support-Level für <code>GatewayClass</code> , <code>Gateway</code> , <code>HTTPRoute</code> , <code>GRPCRoute</code> , <code>TCPRoute</code> , <code>UDPRoute</code> , <code>TLSRoute</code> und <code>ReferenceGrant</code> .
APISIX Ingress Controller nutzt <code>controllerName</code> zur Zuordnung einer <code>GatewayClass</code> .	Apache APISIX Ingress Controller: Configuration dokumentiert <code>controller_name</code> und das passende <code>GatewayClass.spec.controllerName</code> .
APISIX kann über Helm inklusive Ingress Controller installiert und mit Gateway API genutzt werden.	Apache APISIX Ingress Controller: Get APISIX and APISIX Ingress Controller beschreibt Installation mit Gateway API CRDs und Ingress Controller.
Normales Kubernetes Service Routing und DNS-basierte Service Discovery bleiben unabhängig vom Gateway API Ingress-Modell.	Kubernetes Services, Load Balancing, and Networking beschreibt das Kubernetes Netzwerkmodell. Kubernetes DNS for Services and Pods beschreibt Service-DNS und interne Namensauflösung.
Envoy kann Request Header hinzufügen oder entfernen; Header-Verhalten muss daher explizit konfiguriert und geprüft werden. <code>X-Forwarded-*</code> Header dürfen nur mit klar definierten Trusted-Hops/Trusted-CIDRs ausgewertet werden.	Envoy HTTP route configuration und Envoy HTTP route components dokumentieren <code>request_headers_to_add</code> und <code>request_headers_to_remove</code> . Envoy HTTP header manipulation: X-Forwarded-For beschreibt Trusted Client Address, <code>xff_num_trusted_hops</code> und Spoofing-Risiken. Envoy XFF original IP detection dokumentiert Trusted-CIDR-basierte Auswertung.
JWTs sind ein standardisiertes Claims-Format mit Claims wie <code>iss</code> , <code>sub</code> , <code>aud</code> , <code>exp</code> , <code>iat</code> und <code>jti</code> . JWTs müssen sicher validiert werden; Signatur allein reicht nicht aus.	RFC 7519: JSON Web Token definiert JWT und registrierte Claims.
OAuth2 Client Credentials ist das passende Grundmuster für technische Service-Identitäten ohne Benutzerkontext.	RFC 8725: JSON Web Token Best Current Practices beschreibt sichere JWT-Verwendung, Algorithmusprüfung, Claim-Validierung und Cross-JWT Confusion Risiken. RFC 6749: OAuth 2.0 Authorization Framework definiert den Client Credentials Grant. Keycloak OIDC documentation: Client credentials beschreibt die Nutzung in Keycloak.
Token Exchange / On-Behalf-Of ist ein standardisiertes Muster, um ein Token gegen ein anderes Token mit anderer Ziel-Audience oder reduziertem Scope zu tauschen.	RFC 8693: OAuth 2.0 Token Exchange definiert Token Exchange inklusive Delegation und Impersonation. Keycloak: Configuring and using token exchange beschreibt Keycloaks Standard Token Exchange und empfiehlt die unterstützte V2-Variante für interne Token-zu-Token-Exchanges.
Access Tokens sollten für konkrete Zielressourcen/Audiences eingeschränkt werden.	RFC 8707: Resource Indicators for OAuth 2.0 beschreibt Resource Indicators und Audience Restriction. RFC 9068: JWT Profile for OAuth 2.0 Access Tokens beschreibt interoperable JWT Access Tokens für OAuth2 Resource Server.
Moderne OAuth2-Sicherheit verlangt restriktive Flows, saubere Client-Authentifizierung und aktuelle Best Practices.	RFC 9700: Best Current Practice for OAuth 2.0 Security aktualisiert die OAuth2 Security Guidance und deprecates unsichere Betriebsmodi.

Aussage	Belege
API-Gateways und Service-Mesh-Komponenten adressieren unterschiedliche, aber überlappende Sicherheitsanforderungen in Microservice-Architekturen. Autorisierung in Microservices sollte Identität und Attribute von User, Service, Device und Ressource berücksichtigen.	NIST SP 800-204A beschreibt sichere Microservices mit Service-Mesh-Architektur, inklusive Authentication, Authorization, STS, Service Discovery und Monitoring. NIST SP 800-204B beschreibt ABAC für Microservices und Service Mesh, inklusive Zero-Trust-Zielbild und Policy Enforcement.
APISIX kann OIDC/JWT-basierte API-Absicherung und Rate Limiting über Plugins umsetzen.	APISIX openid-connect Plugin dokumentiert OIDC-Integration mit IdPs wie Keycloak. APISIX jwt-auth Plugin dokumentiert JWT-basierte Authentifizierung. APISIX limit-count Plugin dokumentiert Rate Limiting.

Quellenbewertung

Die verwendeten Quellen sind bewusst keine Blogposts:

- IETF RFCs für OAuth2, JWT, Token Exchange, Resource Indicators und Security Best Practices.
- Offizielle Kubernetes Gateway API und Kubernetes Networking Dokumentation.
- Offizielle Apache APISIX Dokumentation.
- Offizielle Keycloak Dokumentation.
- NIST Special Publications zu Microservices Security, Service Mesh und ABAC.

Die offenen Punkte für einen echten PoC bleiben implementierungsspezifisch:

- Unterstützt die konkrete Kubernetes-Distribution mehrere **GatewayClass** Implementierungen im Supportmodell?
- Welche Envoy-Konfiguration ist erforderlich, damit Auth-Header erhalten bleiben und Forwarding-Header korrekt normalisiert werden?
- Welche Keycloak-Version und welche Token-Exchange-Variante stehen im Zielbetrieb zur Verfügung?
- Welche Claims werden verbindlich für Mandant, Kunde, Rollen, Scopes und Audience modelliert?
- Welche Services validieren Tokens selbst und welche Prüfungen übernimmt APISIX?

Talking Points für Cloud Ops

1. Kein Ersatz von Envoy

APISIX ersetzt Envoy nicht global. Envoy bleibt der plattformseitige Standard-Ingress und die einzige öffentliche Eingangsschicht.

2. Keine zweite öffentliche Front Door

APISIX wird intern hinter Envoy betrieben. Externe DNS-Einträge, TLS und Edge Security bleiben bei Envoy.

3. Klare Verantwortlichkeiten

Envoy ist für Plattform-Ingress zuständig. APISIX ist für API-Management-Funktionen zuständig, die näher an den API-Produkten liegen.

4. Gateway API unterstützt mehrere Implementierungen

Gateway API ist dafür ausgelegt, mehrere GatewayClasses zu erlauben. Workloads können explizit wählen, ob sie den Standard-Envoy-Gateway oder den APISIX-Gateway verwenden.

5. Opt-in statt Plattformwechsel

Nur ausgewählte API-Workloads nutzen APISIX. Bestehende Frontends und Standard-Ingress-Routen bleiben unverändert.

6. Bessere API-Governance

APISIX ermöglicht API-spezifische Policies wie Auth Enforcement, Rate Limits, Request Transformationen, Plugin-basierte Erweiterungen und API-spezifische Observability.

7. Keycloak wird zentrale Auth-Quelle

Keycloak stellt Tokens aus. APISIX validiert und erzwingt API-Zugriff. Backend Services behalten fachliche Autorisierung.

8. JWTs unterstützen externe und interne Autorisierung

JWTs dienen als gemeinsamer Identitäts- und Berechtigungskontext für externe Kunden, Benutzer und interne Services. Damit lassen sich kundeneigene Ressourcen konsistent schützen.

9. Interner Traffic bleibt unverändert

Service-zu-Service-Kommunikation über Kubernetes Services wird durch APISIX nicht verändert. JWT-Validierung in Services ergänzt die bestehende Routing-Ebene, ersetzt sie aber nicht.

10. Sicherheitskontrollen bleiben zentral

Die öffentliche Sicherheitskontrolle bleibt bei Envoy. APISIX wird zusätzlich durch NetworkPolicies, RBAC und interne Service-Grenzen abgesichert.

11. Fachliches RBAC gehört in die Services

APISIX prüft API-nahe Policies. Die Entscheidung, ob eine konkrete Kundenressource gelesen oder geändert werden darf, bleibt bei den Backend Services.

12. PoC statt Grundsatzentscheidung

Der Vorschlag ist zunächst ein technischer PoC. Ziel ist eine belastbare Entscheidung auf Basis von Routing, Security, Betrieb, JWT-Nutzung und API-Policy-Anforderungen.